

10 Pearls Project Report

AQI Prediction System

Name: Muhammad Affan Mushahid

Email: affan.mushahid.1@gmail.com

GitHub Repository Link: <https://github.com/Affan-Mushahid/aqi-prediction-system>

1. Introduction

Air quality has a profound impact on public health and environmental safety. The primary objective of this project is to build an automated, end-to-end Machine Learning system capable of predicting the Air Quality Index (AQI) for the next three days for Karachi city. By forecasting pollution levels in advance, this system serves as an early warning tool, allowing individuals and local authorities to take preventative measures against hazardous air quality.

2. Technology Stack

To ensure the system is scalable, lightweight, and fully automated, the project was developed using a modern, serverless technology stack.

- **Data Fetching:** To fetch hourly weather and air quality data for Karachi city, Open-Meteo API was used. Open-Meteo was also used to backfill initial data for the last year.
- **Machine Learning:** Utilizing Python libraries (such as Scikit-Learn and XGBoost) to train, evaluate, and interpret 5 predictive models, namely Random Forest Regression, Gradient Boosting, XGBoost, Support Vector Regressor, and Ridge Regression.
- **Frontend:** Streamlit was used as the frontend to construct an interactive, user-friendly web dashboard for visualizing real-time and forecasted AQI trends.
- **Backend:** FastAPI was used as the backend to build RESTful APIs that connect the machine learning models and database to the user interface.
- **Database:** For this project, MongoDB acts as the Model Registry and Feature Store, handling the flexible storage of raw model data and engineered machine learning features.
- **Automation:** GitHub Actions serves as the CI/CD pipeline engine, automating scheduled tasks such as hourly data fetching and daily model retraining.
- **Deployment:** The frontend and backend were deployed on Render, allowing anyone to visit and view the AQI predictions.

3. Exploratory Data Analysis

3.1. Data Collection & Preprocessing

Using the Open-Meteo API, raw air quality features including 'PM 2.5', 'PM 10', 'dust', 'Carbon Monoxide', 'Carbon Dioxide', 'Nitrogen Dioxide', 'Sulfur Dioxide', 'Ozone', and 'UV Index' were fetched. Weather features including 'Temperature (2 meters)', 'Relative Humidity (2 meters)', 'Wind Speed (10 meters)', 'Wind Direction (10 meters)', and 'Cloud Cover' were also fetched. For the target, 'US AQI' was also fetched.

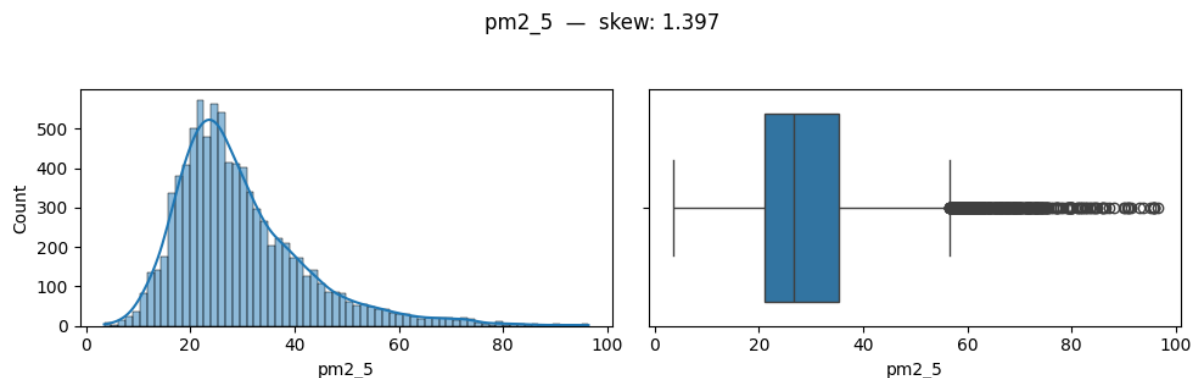
Immediately after fetching the raw data, we fill in any potential missing values using forward fill and linear interpolation for long gaps.

3.2. Univariate Analysis

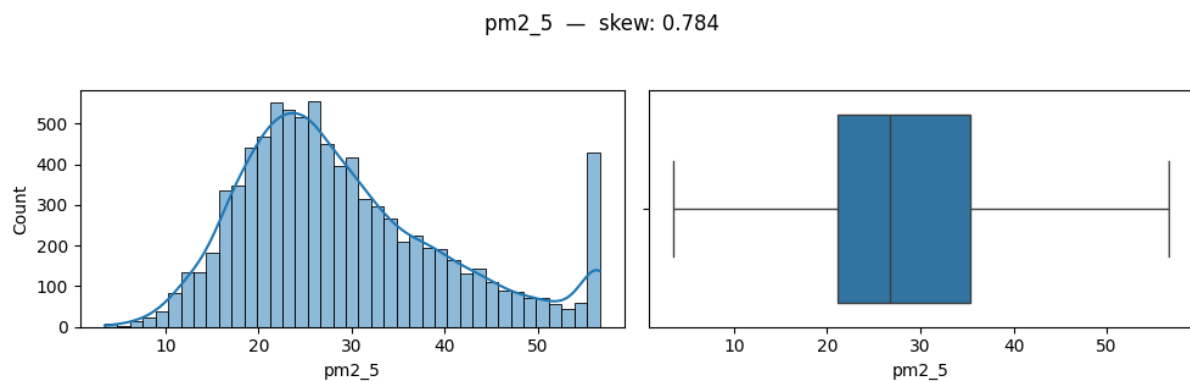
3.2.1. Interquartile Range (IQR) Outlier Handling

Plotting a histogram for each of the fetched features, showed a lot of features had many outliers and heavily skewed. To fix these issues, Interquartile Range (IQR) outlier handling was performed to cap the outliers. Values exceeding 1.5 times the IQR threshold were capped.

Histogram of PM 2.5 before IQR Outlier Handling,



Histogram of PM 2.5 after IQR Outlier Handling,



3.3. Feature Engineering

3.3.1. Temporal & Time-Based Feature

Air quality exhibits strong seasonal and diurnal (daily) patterns. To capture these cyclical trends, several temporal features were extracted, including 'Hour', 'Day of Week', and 'Month'. Cyclical Encoding was also performed on these features.

3.3.2. Wind Components

The wind direction feature was broken down into its u and v components to resolve the mathematical discontinuity at 360 degrees and 0 degrees.

3.3.3. Rolling Means and Standard Deviations

36 hour rolling means were created for 'PM 2.5', 'PM 10', and 'Carbon Monoxide' features. 72 hour standard deviations were also created for 'PM 2.5', and 'PM 10'. These features were chosen after various experimentations and correlation tests, resulting in the best correlation to the target variable.

3.3.4. Derived Features

'AQI Change Rate' was created to find the rate of change of AQI every hour.

'PM Ratio' was created to find the type of pollution in the air.

'Precipitation Likelihood' was created to get a close approximation of the probability of precipitation.

'Is Rush Hour' was a flag created to indicate if it is rush hour (between 7 to 9 AM and 5 PM to 7 PM).

'Is Workday' was a flag created to indicate if it a work day (Monday to Friday).

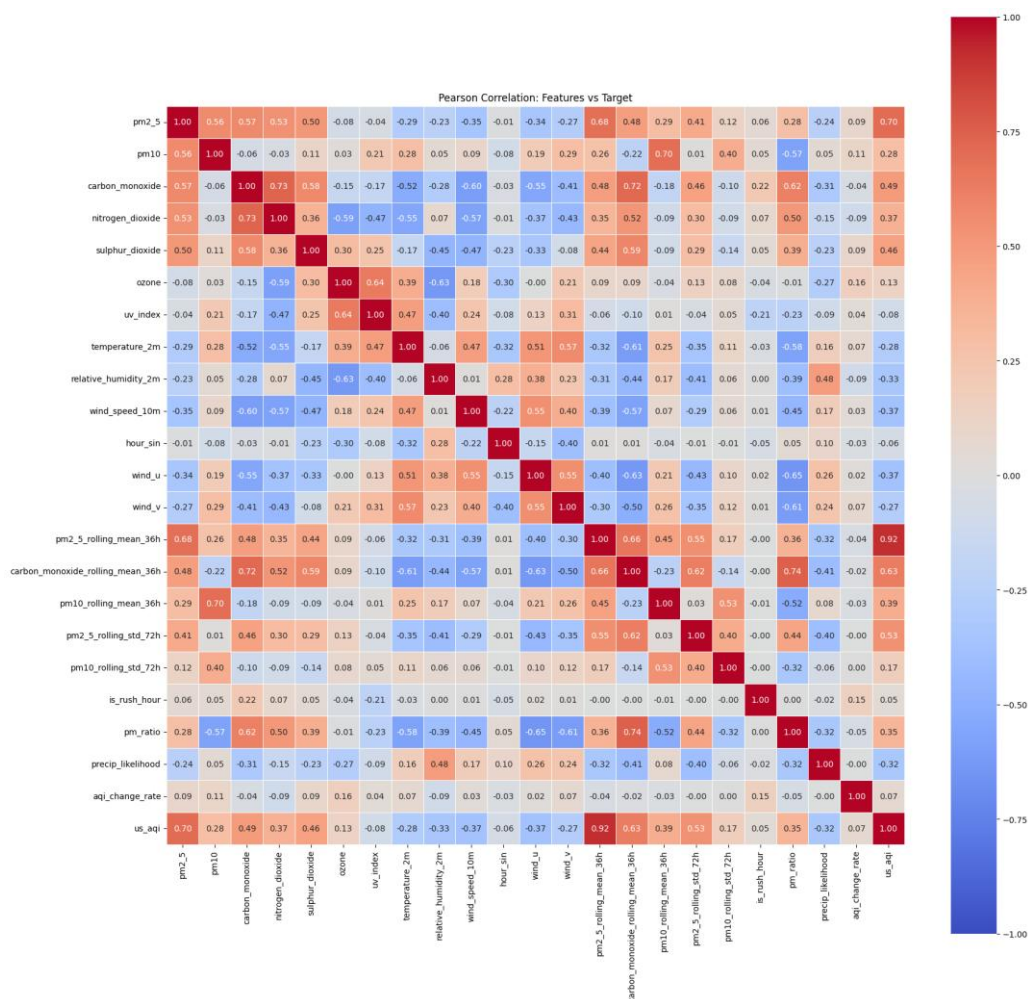
3.4. Multivariate Analysis

3.4.1. Drop Low Target Correlation & High Multicollinearity Features

Due to very low target correlation (less than 0.05) features including 'Dust', 'Day of Week Sin', 'Day of Week Cos', 'Hour Cos', 'Month Sin', and 'Is Workday' were dropped.

Due to high multicollinearity (greater than 0.85), features such as 'Carbon Dioxide', 'Cloud Cover', and 'Month Cos' were dropped.

3.4.2. Correlation Heatmap



4. Model Building

4.1. Model Selection

For this project, 5 predictive models were trained and evaluated, namely, 'Random Forest Regression', 'Gradient Boosting', 'XGBoost', 'Support Vector Regressor', and 'Ridge Regression'.

4.2. Model Evaluation

After training each model, they are evaluated based on their R2 score, followed by RMSE and finally the MAE.

For the given dataset, 'XGBoost' showed the best performance with an R2 score of: 0.85. The complete table for each model is as follows,

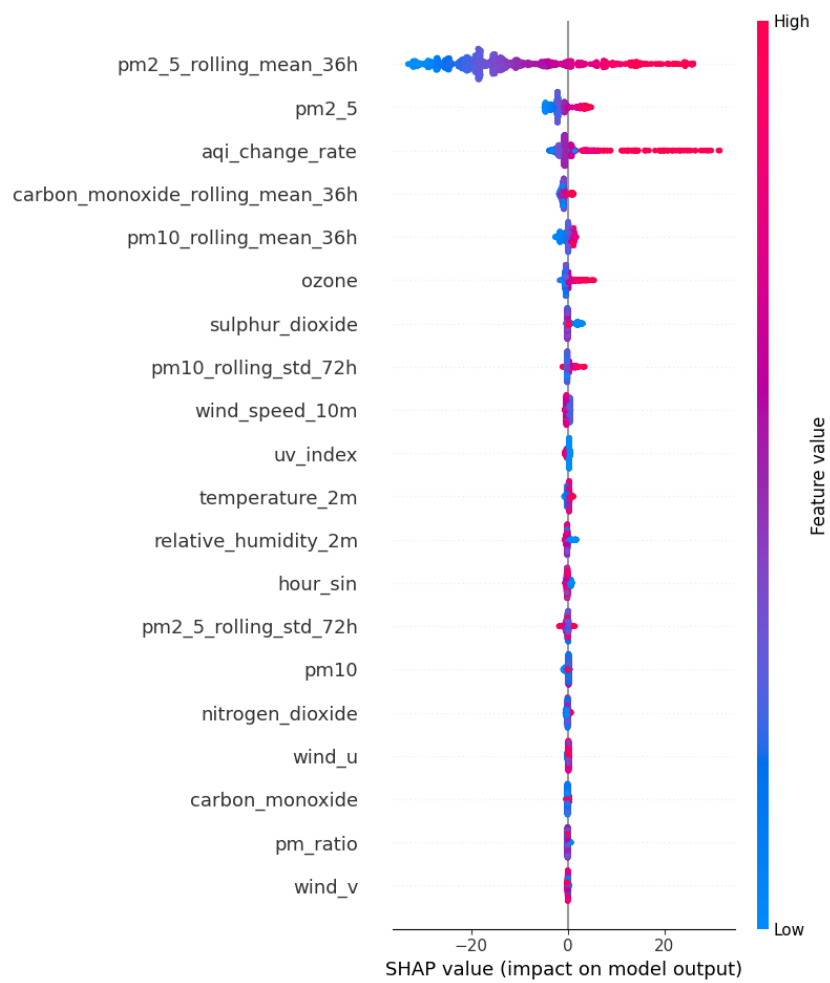
Name	R2	RMSE	MAE
XGBoost	0.850	7.410	4.653
Random Forest Regressor	0.834	7.773	4.822
Ridge Regression	0.828	7.925	5.040
Gradient Boosting	0.828	7.931	5.202
Support Vector Regressor	0.509	13.387	10.347

4.4. Model Explainability

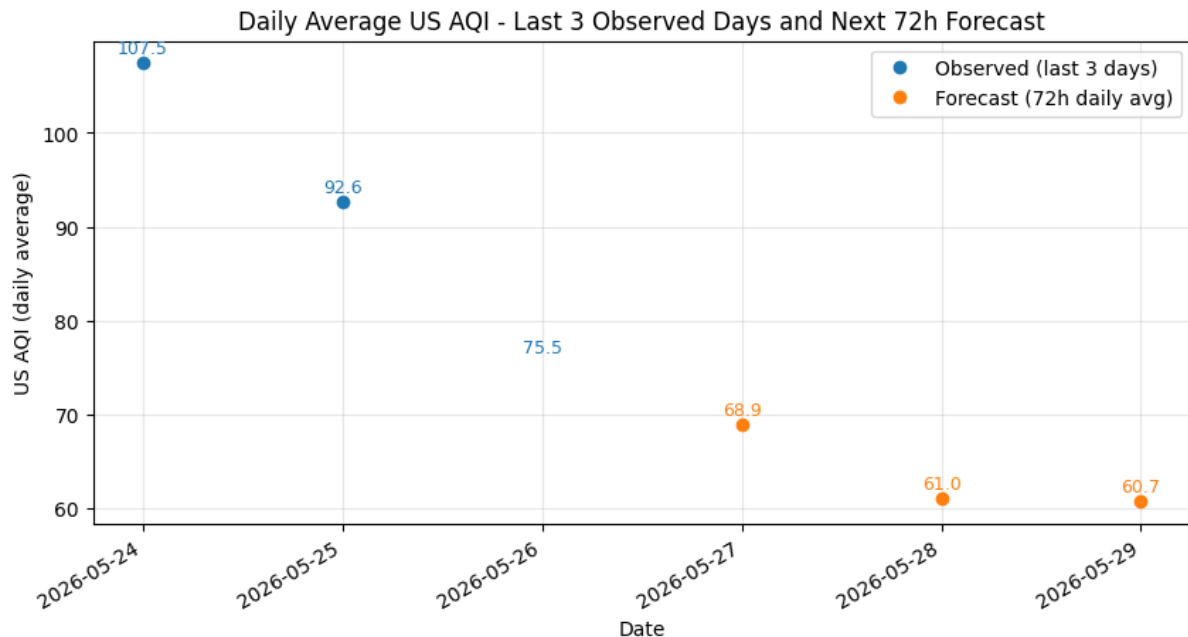
SHAP (SHapley Additive exPlanations) was performed on the XGBoost model to explain the reasoning behind its answer.

Using SHAP, we can see that,

- Historical PM 2.5 is the dominant driver. High values of the 36-hour rolling mean strongly increase the model's output. Conversely, low values strongly decrease the prediction.
- PM 2.5 is the second important feature, behaving similarly to the rolling mean but with a slightly narrower impact range.
- AQI Change Rate is another important feature, a sudden spike in AQI change rate acts as a strong warning signal for a high final prediction.
- Features like pm10_rolling_mean_36h, carbon_monoxide_rolling_mean_36h, and ozone have moderate, predictable impacts. High values generally lead to higher predictions, but their maximum impact is much smaller than PM 2.5.



4.5. Sample Predictions



5. Database Storage

For this project, MongoDB was utilized as the primary database. It serves a crucial dual purpose in the architecture,

Feature Store: Storing the processed, engineered features and target variables. By acting as a centralized feature store, MongoDB allows the scheduled model-training scripts and the FastAPI inference endpoints to rapidly query the exact data they need.

Model Registry: MongoDB was also used for securely storing the trained models which performed the best into the database.

6. CI/CD Pipeline & Automation

For this project, GitHub Actions workflows were utilized to automate the feature and training pipelines.

6.1. Feature Pipeline

Every hour, the pipeline fetches the latest raw data from Open-Meteo API, cleans it, performs feature engineering and then stores the engineered features into the MongoDB database.

6.2. Training Pipeline

Every day, the model fetches the latest data from the MongoDB feature store, trains the models, evaluates the best model and then stores the best model into the model registry on the MongoDB database.

7. User Interface

To provide an easy-to-use interface for the end user, Streamlit was utilized for a simple and straight-forward interface which shows users the current AQI prediction and the prediction for the next 3 days. Users can also see the last 24 hour values of all raw variable and they can also see the SHAP to get a good idea and explanation of how the model is performing the predictions.

8. Backend Server

FastAPI was selected as the backend framework for the AQI Prediction System because it provides a high-performance, asynchronous bridge between the computationally intensive machine learning pipelines and the interactive user interface.

9. Deployment

To allow users to interact with the project and view it, the project, including the frontend and backend was deployed on Render, allowing a simple transition from local development to a live, publicly accessible dashboard.

10. Challenges and Issues

10.1. Feature Selection

A big task of the project was feature selection. A lot of times when selecting features it was very common to accidentally introduce data leakage or cause overfitting. Features such as aqi lag 1 hour caused the models to severely overfit leading to great train accuracy but completely failing when given a test set. Such features were dropped.

10.2. Database Selection

Initially Hopsworld was chosen as the feature store and model registry. But due to several instability and materialization issues, it was replaced with MongoDB as both the feature store and Model Registry.

10.3. Naive vs Aware Datetimes

There were incompatibilities and failures due to some times being naive datetime objects and others being aware, it caused many comparisons to fail. Also, issues arrived when the features were fetched from MongoDB database, as these were stored as aware datetime but while being fetched they were being fetched naive datetime objects. This caused large incompatibilities because their time would not represent the actual time. These incompatibilities were fixed by ensuring all timestamps were in UTC and converted to aware datetime objects while storing and retrieving.

10.4. Model Hyperparameter Selection

Various models seemed like they were either overfitting or underfitting, the issue was due to the non-optimal hyperparameters being set. Fixing this required a lot of experimentation to find the optimal hyperparameters for each model.

10.5. Deployment Issues

Render was chosen as the platform for deployment. The free tier of Render shuts down servers after some time. So, if someone tried to visit the site after it would shut down, the frontend would wake up, but the backend server was unable to start back up. Fixing this required setting larger timeouts to ensure server would wake up, and instructions were also implemented instructing users to manually wake up the backend if the server did not wake up in time.